



Jarosław Kuchta

Jakość Oprogramowania

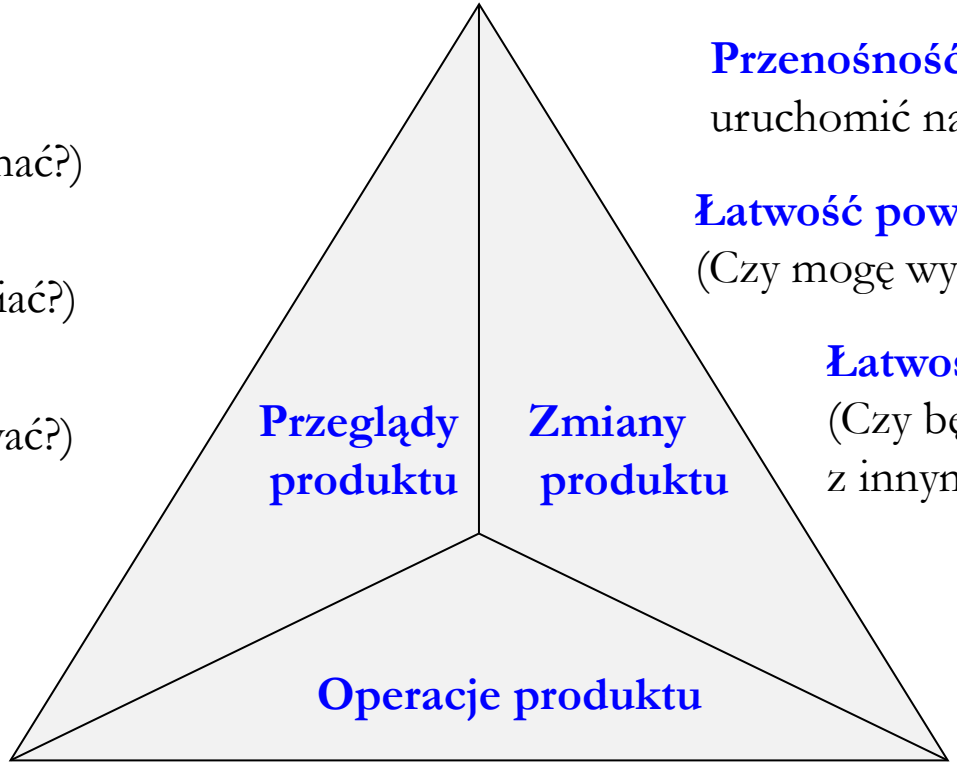
Model jakości McCalla

Czynniki jakości produktu

Utrzymywalność
(Czy mogę to utrzymać?)

Elastyczność
(Czy mogę to zmieniać?)

Testowalność
(Czy mogę to testować?)



Przenośność (Czy mogę to uruchomić na innej maszynie?)

Łatwość powtórnego użycia
(Czy mogę wykorzystać część tego?)

Łatwość współdziałania
(Czy będę mógł współpracować z innym systemem?)

źródło: Pressman

Poprawność (Czy to robi to, co ja chcę?)
Wiarygodność (Czy to robi dokładnie to przez cały czas?)
Wydajność (Czy to będzie odpowiednio działać na moim sprzęcie?)
Integralność (Czy to jest bezpieczne?)
Użyteczność (Czy to jest zaprojektowane pod użytkownika?)

Czynniki jakości (1/3)

- **Poprawność (*correctness*)** - stopień zgodności programu ze specyfikacją i zaspokojenia celów przedsięwzięcia
- **Wiarygodność (*reliability*)** - stopień, w jakim program wykonuje zamierzone funkcje z wymaganą precyzją
- **Wydajność (*efficiency*)** - ilość przetwarzanych zasobów i kodu, które program potrzebuje do wykonywania swoich funkcji
- **Integralność (*integrity*)** - stopień odporności programu lub danych na nieuprawnione użycie lub modyfikację
- **Użyteczność (*usability*)** - wysiłek, który musi być włożony w nauczenie się programu, w jego użycie, przygotowanie danych wejściowych i interpretację danych wyjściowych

Czynniki jakości (2/2)

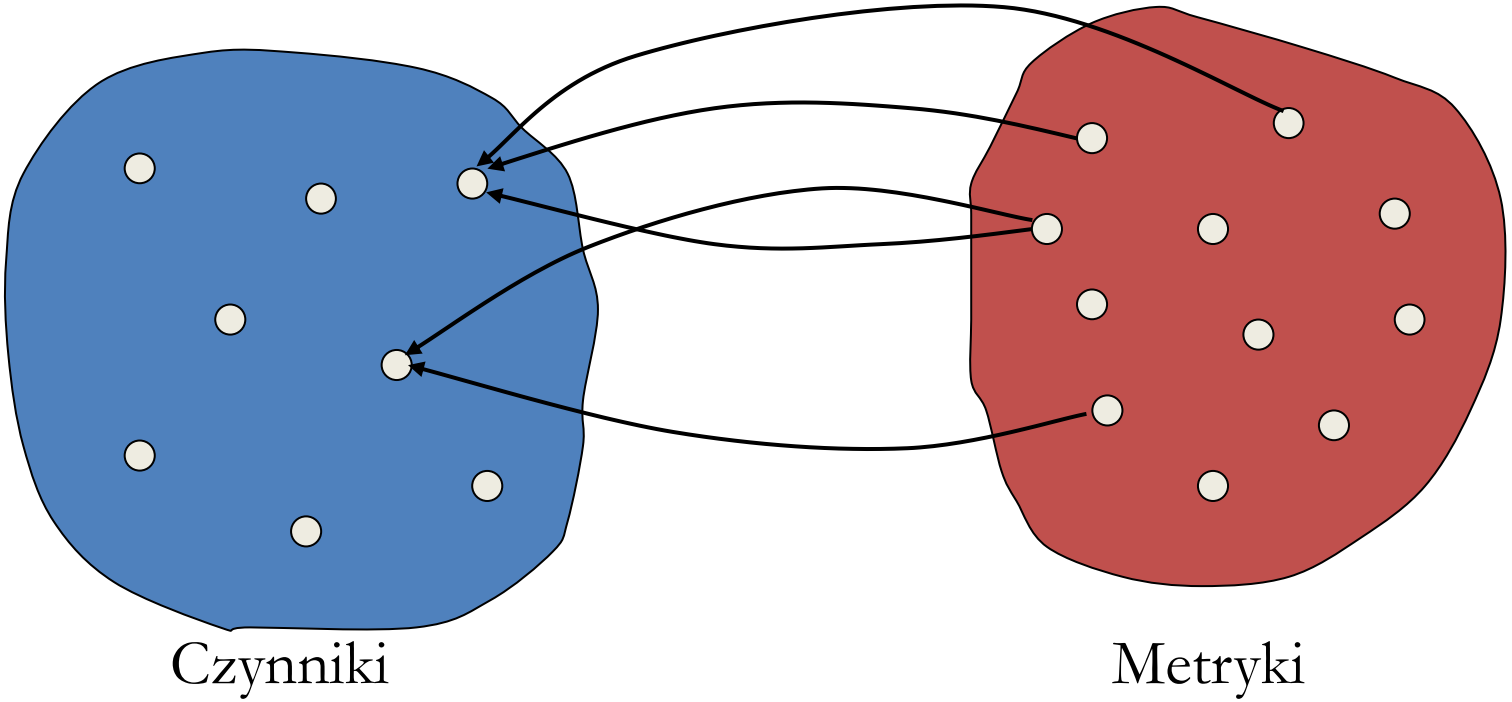
- **Łatwość pielęgnacji (*maintainability*)** - wysiłek, który musi być włożony w lokalizację błędów i ich naprawę
- **Elastyczność (*flexibility*)** - wysiłek wymagany dla modyfikacji działającego programu
- **Testowalność (*testability*)** - wysiłek, który jest potrzebny dla upewnienia się, że program funkcjonuje poprawnie

Czynniki jakości (3/3)

- **Przenośność (*portability*)** - wysiłek, który może być wymagany dla przeniesienia oprogramowania z jednego systemu sprzętowego lub programowego do drugiego
- **Możliwość powtórnego wykorzystania (*reusability*)** - stopień, w jakim całość lub część oprogramowania może zostać powtórnie wykorzystana w innej aplikacji
- **Łatwość współdziałania (interoperability)** - wysiłek, który musi być włożony dla podłączenia jednego systemu do drugiego

Czynniki a metryki

$$F_q = c_1 \times m_1 + c_2 \times m_2 + \ldots + c_n \times m_n$$



Kryteria jakości (1)

- Łatwość śledzenia (Traceability)
  - Powiązania od wymagań do implementacji z uwzględnieniem środowiska wytwarzania i wykonania
- Kompletność (Completeness)
  - Pełna implementacja wymaganych funkcji
- Spójność (Consistency)
  - Jednolite techniki i notacje projektowania i implementacji
- Dokładność (Accuracy)
  - Zapewniona wymagana precyzja obliczeń i wyników
- Odporność na błędy (Error tolerance)
  - Ciągłość działania poza nominalnymi warunkami.
- Prostota (Simplicity)
  - Implementacja funkcji w sposób jak najbardziej zrozumiały (zazwyczaj z unikaniem praktyk, które zwiększają złożoność)
- Modularność (Modularity)
  - Struktura złożona z wysoce niezależnych modułów

Kryteria jakości (2)

- Ogólność (Generality)
  - Szeroki zakres wykonywanych funkcji
- Rozszerzalność (Expandability)
  - Możliwość rozszerzenia wymagań dotyczących przechowywania danych lub funkcji obliczeniowych
- Instrumentacja (Instrumentation)
  - Pomiar użycia oprogramowania oraz identyfikacja błędów
- Samoopisowość (Self-descriptiveness)
  - Wyjaśnienie implementacji funkcji
- Efektywność wykonania (Execution Efficiency)
  - Możliwie jak najkrótszy czas wykonania
- Efektywność przechowywania danych (Storage Efficiency)
  - Możliwie jak najmniejsze zapotrzebowanie na pamięć w czasie wykonania
- Kontrola dostępu (Access Control)
  - Kontrola dostępu do oprogramowania i danych
- Monitorowanie dostępu (Access Audit)
  - Sprawdzanie dostępu do programów i danych

Kryteria jakości (3)

- Operatywność (Operability)
  - Operacje i procedury ułatwiające korzystanie z interfejsu użytkownika
- Łatwość nauki (Training)
  - Ułatwienie przejścia od wstępnej znajomości oprogramowania do sprawnego wykorzystywania
- Komunikatywność (Communicativeness)
  - Akceptowanie danych wyjściowe i generowanie danych wyjściowych w sposób, który jest łatwy do stosowania
- Niezależność od oprogr. systemowego
- (Software System Independence)
  - Niezależność oprogramowania od środowiska programowego (systemu operacyjnego, narzędzi, procedur wejścia/wyjścia itp.)
- Niezależność sprzętowa (Machine Independence)
  - Niezależność oprogramowania od infrastruktury sprzętowej
- Ujednolicenie komunikacji (Communications Commonality)
  - Stosowanie standardowych protokołów i procedur interfejsowych
- Ujednolicenie danych (Data Commonality)
  - Stosowanie standardowych reprezentacji danych
- Zwartość (Conciseness)
  - Implementacja funkcji poprzez najkrótszy możliwy kod

Wpływ kryteriów na czynniki (1)

	Poprawność	Wiarygodność	Wydajność	Integralność	Użyteczność	Łatwość pielęgnacji	Łatwość testowania	Elastyczność	Przenośność	Łatwość powtórnego użycia	Łatwość współdziałania
Łatwość śledzenia	+					+	+	+		+	
Kompletność	+	+			+						
Spójność	+	+				+	+			+	
Dokładność		+	✖		+						
Odporność na błędy	+	+	✖		+						
Prostota	+	+	+			+	+	+	+	+	
Modularność			✖			+	+	+	+	+	+
Ogólność		✖	✖	✖				+		+	+
Rozszerzalność			✖					+		+	
Instrumentacja			✖		+	+	+				
Samoopisowość			✖			+	+	+	+	+	

Wpływ kryteriów na czynniki (2)

	Poprawność	Wiarygodność	Wydajność	Integralność	Użyteczność	Łatwość pielęgnacji	Łatwość testowania	Elastyczność	Przenośność	Łatwość powtórnego użycia	Łatwość współdziałania
Efektywność wykonania			+						✖		
Efektywność przechowywania danych			+				✖		✖		
Kontrola dostępu			✖	+	+			✖			✖
Monitorowanie dostępu			✖	+							
Operatywność			✖		+					+	
Łatwość nauki					+					+	
Komunikatywność			✖		+	+	+	+		+	
Niezależność od oprogr. systemowego			✖					+	+	+	+
Niezależność sprzętowa			✖					+	+	+	+
Ujednolicenie komunikacji											+
Ujednolicenie danych				✖						+	+
Zwartość	+		+			+	+				

Metryki i miary (1)

	Analiza wymagań	Projekt	Imple-mentacja
TT. Łatwość śledzenia			
TR1. Odwołania z modułów do wymagań		●	●
CP. Kompletność			
CP1. Lista kontrolna dla kompletności	●	●	●
CS1. Spójność			
CS1. Spójność procedur		●	●
CS2. Spójność danych		●	●
AY. Dokładność			
AY1. Lista kontrolna dla dokładności	●	●	●
ET. Odporność na błędy			
ET1. Kontrola błędów		●	●
ET2. Odzyskiwanie (sprawności) po niepoprawnych danych	●	●	●
ET3. Odzyskiwanie (sprawności) po awarii obliczeniowej	●	●	●
ET4. Odzyskiwanie (sprawności) po awarii sprzętowej	●	●	●
ET5. Odzyskiwanie (sprawności) po awarii urządzeń	●	●	●
SI. Prostota			
SI1. Prostota struktury projektowej		●	●
SI2. Stosowanie języka lub preprocesora strukturalnego			●
SI3. Złożoność modułów		●	●
SI4. Złożoność kodowania			●

Metryki i miary (2)

	Analiza wymagań	Projekt	Imple-mentacja
MO. Modularność			
MO1. Stabilność (siła i powiązanie modułów)		●	
MO2. Implementacja modularna		●	●
GE. Ogólność			
GE1. Stopień odwołań innych modułów do danego modułu		●	●
GE2. Ogólność implementacji		●	●
EX. Rozszerzalność			
EX1. Rozszerzalność przechowywania danych		●	●
EX2. Rozszerzalność kodu		●	●
IN. Instrumentacja			
IN1. Testowalność modułów		●	●
IN2. Testowalność integracyjna		●	●
IN3. Testowalność systemowa		●	●
SD. Samoopisowość			
SD1. Liczność komentarzy			●
SD2. Efektywność komentarzy			●
SD3. Samodokumentacja kodu implementacji			●
EE. Efektywność wykonania			
EE1. Przypisanie wymagań dot. wydajności do projektu		●	
EE2. Efektywność przetwarzania iteracyjnego		●	●
EE3. Efektywność wykorzystania danych		●	●
SE. Efektywność przechowywania danych			
SE1. Efektywność przechowywania danych		●	●

Metryki i miary (3)

	Analiza wymagań	Projekt	Imple-mentacja
AC. Kontrola dostępu			
AC1. Lista kontrolna dla kontroli dostępu	●	●	●
AA. Monitorowanie dostępu			
AA1. Lista kontrolna dla monitorowania dostępu	●	●	●
OP. Operatywność			
OP1. Lista kontrolna dla operatywności	●	●	●
TC. Łatwość nauki			
TC1. Lista kontrolna dla łatwości nauki		●	●
CM. Komunikatywność			
CM1. Komunikatywność interfejsu wejściowego użytkownika		●	●
CM2. Komunikatywność interfejsu wyjściowego użytkownika		●	●
SS. Niezależność od oprogramowania systemowego			
SS1. Niezależność od systemu op. i języka prog.		●	●
MI. Niezależność sprzętowa			
MI1. Niezależność od maszyny		●	●
CC. Ujednolicenie komunikacji			
CC1. Lista kontrolna dla ujednolicenia komunikacji	●	●	●
DC. Ujednolicenie danych			
DC1. Lista kontrolna dla ujednolicenia danych	●	●	●
CO. Zwartość			
CO1. Miara Halstead ’a			●

Współczesne propozycje metryk

- Z punktu widzenia współczesnego użytkownika
- Pytania tak/nie (wartości „1” i „0”)
- Uśrednianie odpowiedzi dla obliczenia metryk
- Wskazówki dla projektantów i programistów

Łatwość śledzenia (Traceability)

- Czy działanie programu jest monitorowane w logach?
- Czy istnieje możliwość wygenerowania i sprawdzenia dokumentów pośrednich, tymczasowych?

Operatywność (Operability)

- Czy liczba kroków (kliknięć myszą lub naciśnięć klawiszy) prowadzących przez interfejs użytkownika do często wykonywanych operacji jest odpowiednia?
- Czy stosuje się menu kontekstowe dla ułatwienia dostępu do komend odpowiednich dla wybranego obiektu?
- Czy stosuje się przyciski narzędziowe dla często wykonywanych operacji?
- Czy istnieją skróty klawiszowe do najczęściej wykonywanych operacji?
- Czy istnieje możliwość dostosowania przycisków narzędziowych?
- Czy istnieje możliwość dostosowania skrótów klawiszowych?
- Czy istnieje możliwość dostosowania menu?
- Czy stosuje się podpowiedzi przy wprowadzaniu danych?
- Czy możliwe jest powtarzanie operacji?
- Czy możliwe jest grupowanie operacji?
- Czy istnieje możliwość automatyzacji pracy?

Dokładność (Accuracy)

- Czy zastosowano odpowiednią precyzję obliczeń?
- Czy odpowiednio określono zakresy danych?

Kompletność (Completeness)

- Czy funkcje, które deklarują twórcy (producent) oprogramowania są w pełni zaimplementowane?
- Czy oprogramowanie ma wszystkie funkcjonalności oferowane przez inne programy w swojej kategorii?
- Czy nie brakuje w oprogramowaniu funkcji, które są dla mnie szczególnie potrzebne?
- Czy mam możliwość dodania szczególnych funkcji, których mi brakuje?

Rozszerzalność (Expandability)

- Czy użytkownik może rejestrować sekwencje własnych działań do wielokrotnego użycia?
- Czy istnieje możliwość pisania własnych funkcji (skryptów)?
- Czy można dodawać wywołania własnych funkcji (skryptów) do standardowego interfejsu użytkownika?
- Czy są dostępne dodatki (ang. *add-ins*) do programu pisane przez innych?
- Czy program ma specjalne mechanizmy wspierające pracę z dużymi dokumentami, które nie mieszczą się w pamięci operacyjnej?
- Czy istnieje możliwość włączenia do dokumentu obiektów zarządzanych przez inne aplikacje (np. typu COM)?

Spójność (Consistency)

- Czy interfejs systemu w różnych jego komponentach ma taką samą formę?
- Czy w całym systemie używa się tej samej formy wprowadzania danych?
- Czy możliwe jest przejście od jednego komponentu systemu do drugiego jest możliwe bez wychodzenia z systemu?
- Czy interfejs systemu jest podobny do interfejsu innych systemów tej samej klasy?

Prostota (Simplicity)

- Czy funkcje programu są proste i zrozumiałe?
- Czy struktura interfejsu użytkownika jest prosta?
- Czy ikony na przyciskach są proste i zrozumiałe?
- Czy napisy na elementach interfejsu (np. przyciskach) są proste i zrozumiałe?
- Czy opisy dodatkowe (podpowiedzi, wyjaśnienia, komunikaty) są proste i zrozumiałe?

The Magical Number Seven, Plus Or Minus Two

Odporność na błędy (Error tolerance)

- Czy wprowadzane dane są walidowane?
- Czy użytkownik ma możliwość cofnięcia operacji zmiany danych?
- Czy w programie tworzy się automatycznie kopie zapasowe danych?
- Czy można łatwo sprawdzić odtworzenie danych z kopii zapasowej?
- Czy ilość danych ewentualnie traconych w przypadku awarii jest akceptowalna?
- Czy producent zapewnia wsparcie techniczne dla sytuacji awaryjnych?
- Czy producent publikuje listę najczęściej zadawanych pytań i odpowiedzi?
- Czy producent zbiera informację o błędach od użytkowników?
- Czy producent oferuje bezpłatne nakładki korygujące błędy?
- Czy producent oferuje ewentualne aktualizacje systemu do kolejnych wersji po preferencyjnych cenach?

Ogólność (*Generality*)

- Czy program może mieć zastosowanie w wielu dziedzinach?
- Czy program może mieć zastosowanie w konkretnie potrzebnej dziedzinie?
- Czy część oprogramowania można wykorzystać w innej dziedzinie zastosowania?

Samoopisowość (Self-descriptiveness)

- Czy elementy menu i przyciski mają opisy zrozumiałe dla użytkownika?
- Czy pola danych na dialogach mają etykiety zrozumiałe dla użytkownika?
- Czy ewentualne objaśnienia dla pól danych są przydatne dla użytkownika?

Instrumentacja (Instrumentation)

- Czy program podaje stan, w jakim się znajduje?
- Czy program podaje nazwę dokumentu, nad którym użytkownik pracuje?
- Czy program podaje postęp długotrwałych operacji?
- Czy program przedstawia użytkownikowi listę ostatnio otwieranych dokumentów?
- Czy przy otwieraniu dokumentu system oferuje podgląd jego zawartości?
- Czy program prezentuje podgląd wydruku?
- Czy system przedstawia dokumenty w postaci *What You See Is What You Get*?
- Czy komunikaty o błędach są zrozumiałe?
- Czy komunikaty o błędach wskazują na możliwe przyczyny błędów?
- Czy komunikaty o błędach wskazują na możliwości usunięcia przyczyn lub skutków błędów?

Komunikatywność (Communicativeness)

- Czy dane są podawane w formacie odpowiednim dla użytkownika?
- Czy dane są podawane w jednostkach odpowiednich dla użytkownika?
- Czy dane nie wymagają przeliczania przez użytkownika?
- Czy dane nie wymagają zapamiętywania przez użytkownika?

Efektywność wykonania (Execution Efficiency)

- Czy czas reakcji programu na działania użytkownika jest zadowalający?
- Czy czas reakcji systemu na zdarzenia zewnętrzne jest wystarczający?
- Czy dla długotrwałych operacji zastosowano osobne wątki?

Efektywność przechowywania danych (Storage Efficiency)

- Czy zapotrzebowanie programu na pamięć operacyjną nie jest zbyt duże?
- Czy zapotrzebowanie programu na pamięć dyskową nie jest zbyt duże?

Kontrola dostępu (Access Control)

- Czy zastosowano uwierzytelnianie i autoryzację użytkowników?
- Czy wyróżnia się kategorie (role) użytkowników z różnym poziomem dostępu?
- Czy zastosowano odpowiednie szyfrowanie poufnych danych?
- Czy zablokowano boczne furtki dostępu?

Monitorowanie dostępu (Access Audit)

- Czy wykrywa się próby nieuprawnionego dostępu lub nieautoryzowanej modyfikacji?
- Czy istnieje możliwość odtworzenia danych w przypadku nieuprawnionej modyfikacji?

Łatwość nauki (Training)

- Czy system informuje nowego użytkownika o swoich podstawowych możliwościach?
- Czy interfejs systemu jest zgodny z przyjętymi standardami, w tym z innymi systemami tej samej klasy?
- Czy system zachowuje się w sposób zgodny z intuicją?
- Czy system oferuje pomoc dla użytkownika?
- Czy stosuje się podpowiedzi do menu i przycisków sterowania?
- Czy system dostosowuje się do umiejętności użytkownika?
- Czy system oferuje szablony?
- Czy system oferuje dokumenty przykładowe?
- Czy system jest wyposażony w program demonstracyjny?
- Czy system jest wyposażony w program do samodzielnej nauki (ang. *tutor*)?

Modularność (Modularity)

- Czy system składa się z komponentów, które można niezależnie instalować?
- Czy system składa się z komponentów, które można wykorzystać w innych systemach?

Niezależność od oprogramowania systemowego (Software System Independence)

- Czy program może zostać uruchomiony pod różnymi systemami operacyjnymi?
- Czy istnieją wersje programu dla różnych systemów operacyjnych?
- Czy dane są przenośne między różnymi systemami operacyjnymi?
- Czy sposób kodowania znaków uwzględnia różne języki?
- Czy sposób kodowania liczb i dat uwzględnia różne języki?
- Czy istnieją różne wersje językowe interfejsu użytkownika?

Niezależność sprzętowa (Machine Independence)

- Czy istnieje wersja programu dla danej platformie sprzętowej (np. na smartfon)?
- Czy program da się uruchomić na starszych komputerach?
- Czy system nie wymaga jakiś innych, specjalnych urządzeń zewnętrznych?

Ujednolicenie komunikacji (Communications Commonality)

- Czy do połączenia z innymi systemami stosuje się standardowe interfejsy?
- Czy do połączenia z innymi systemami stosuje się standardowe protokoły?

Ujednolicenie danych (Data Commonality)

- Czy w systemie stosuje się standardowy sposób kodowania znaków (UTF-8)?
- Czy w systemie stosuje się standardowe formaty plików (np. XML)?
- Czy przy wymianie danych z innymi systemami stosuje standardowe formaty (np. Jason)?
- Czy w systemie stosuje się powszechnie dostępne systemy baz danych?

Zwartość

- Czy w programie efektywnie korzysta się z pamięci operacyjnej?
- Czy w programie stosuje się efektywny zapis danych?
- Czy w programie efektywnie korzysta się z kanału komunikacyjnego?

Literatura

- J. A. McCall, P. K. Richards, G. F. Walters: *Factors in Software Quality. Vol.I. Concept and Definitions of Software Quality*, US Rome Air Development Center Reports, 1977
- J. A. McCall, P. K. Richards, G. F. Walters: *Factors in Software Quality. Vol.II. Metric Data Collection and Validation*, US Rome Air Development Center Reports, 1977
- J. A. McCall, P. K. Richards, G. F. Walters: *Factors in Software Quality. Vol.III. Preliminary Hanbook on Software Quality for an Acquisition Manager*, US Rome Air Development Center Reports, 1977
- G. A. Miller: *The magical number seven, plus or minus two: Some limits on our capacity for processing information*, [w] Psychological review, nr 63.2, 1956, s.81-97